Exercice 3.1

Importez dans *IntelliJ* le projet fourni sous forme de fichier ZIP (`sources03.zip`). Il s'agit d'une correction partielle du laboratoire précédent dans laquelle certains bugs ont été introduits.

Pour cela, décompressez le fichier Zip dans votre répertoire personnel puis ouvrez le projet dans *IntelliJ* à l'aide de *File* et *Open...*.

Exécutez la classe `DateTest` en cliquant avec bouton droit sur le fichier `DateTest.java` et en sélectionnant *Run 'DateTest'*.

Vous constaterez que certains tests ne fonctionnent pas. Sans toucher aux tests, corrigez la classe `Date` jusqu'à ce que tous les tests fonctionnent.

Exercice 3.2

Explorez la classe `DateTest`.

Ajoutez-y les tests suivants :

- Valider le fonctionnement du constructeur sans paramètre.
- Valider le fonctionnement de `dayOfYear()` pour une année non bissextile et une année bissextile. Testez particulièrement les cas limites (premier et dernier jour de l'année par exemple).
- Valider le fonctionnement de `dayOfWeek()`.
- Valider le fonctionnement de la méthode statique `isLeapYear()`.

Au besoin, corrigez votre classe `Date` afin que tous les tests fonctionnent.

Exercice 3.3

Ajoutez quelques tests qui permettraient de vérifier que `Date` contient une méthode `equals` qui renvoie vrai si deux dates (à priori deux objets différents) dénotent le même jour.

Ensuite, implémentez cette méthode `equals` dans `Date` et validez vos tests.

Exercice 3.4

Ajoutez des tests pour vos classes `Date` et `Person` pour vérifier qu'elles contiennent une méthode `compareTo` qui les compare. On comparera les dates selon l'ordre naturel et les personnes sur base leur age en jours (c'est à dire l'ordre inverse de leur date de naissance).

Ensuite, implémentez ces méthodes et validez vos tests.

Exercice 3.5

Dans une classe `Program`, écrivez une méthode statique qui reçoit un tableau de `Date` et qui trie ces dates à l'aide de votre méthode `compareTo` et d'un algorithme de tri au choix. Essayez ensuite cette méthode à l'aide de tests unitaires.

Voici un algorithme de tri par sélection qui trie un tableau d'entiers :

```
pour chaque indice I du tableau
  POS <-- la position de l'entier le plus petit dans le tableau à partir de I
  echanger la case I avec la case POS
fin pour
```

Exercice 3.6

Dans la classe de tests de classe `Person`, créez les tests suivants :

- Créer une personne avec une date de naissance impossible déclenche une `RuntimeException`.
- Valider le fonctionnement de la méthode `getAge()` à l'aide de cas limites. *Comme cette méthode est difficile à tester car elle se base sur la date du jour, créez et testez plutôt une méthode d'instance* `getAgeAt(Date d)` *qui renvoie le nombre d'années d'une personne à la date donnée.*

Au besoin, corrigez votre classe `Person` afin que tous les tests fonctionnent.

Exercice 3.7

Ajoutez quelques tests qui permettraient de vérifier que `Person` contient une méthode `equals` qui renvoie vrai si deux personnes (à priori deux objets différents) ont le même nom, le même prénom et une date de naissance identique. Attention, les dates de naissance des deux personnes pourraient être des objets différents qui dénotent la même date.

Ensuite, implémentez cette méthode `equals` dans `Person` et validez vos tests.

Exercice 3.8

Dans la classe `Program`, écrivez une méthode statique qui reçoit un tableau de `Person` et qui trie ces personnes à l'aide de votre méthode `compareTo` et d'un algorithme de tri au choix. Essayez ensuite cette méthode à l'aide de tests unitaires.